8-5 · 데이터 파이프라인·최종 RAG 챗봇 · 실습 1.5H

Airflow DAG + RAG 챗봇 통합 구축

최종 DAG · 챗봇 최종본 · 이름과 차원 대조

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1.5시간 뒤에는

① health · collections · Ollama 세 줄로 전 구성 요소를 전수 점검한다

② @daily · retries 2 · ingested_at 을 갖춘 rag_pipeline_final 을 쓴다

③ 챗봇 사이드바에 조각 수 · 차원, 근거마다 적재 시각을 띄운다

④ 컬렉션 이름 세 곳과 임베딩 차원을 대조한다

사전 준비: 실습점검 1~3단계(전부 올림 · 네트워크 · lecture_kb 11 · 가상환경)

실습 흐름

다섯 단계 — DAG 폴더와 챗봇 폴더를 오가며

~/airflow-lab(DAG) 과 ~/data-eng/week8(챗봇) 두 터미널을 열어 둡니다 — af 함수는 DAG 쪽에

1. 전수 점검	2. 최종 DAG	3. 챗봇 최종본	4. 이름 대조	5. 차원 대조
monitor/health collections · Ollama	rag_pipeline_final @daily · retries 2	조각 수 · 차원 적재 시각	grep lecture_kb 세 곳	"size":768 embed_query 768

1단계

전수 점검 — 셋 다 통과해야 출발

1. **curl -s http://localhost:8080/api/v2/monitor/health**
   metadatabase · scheduler · triggerer · dag_processor 모두 "status":"healthy" (실측)
2. **curl -s http://localhost:6333/collections**
   목록에 lecture_kb
3. **curl -s http://localhost:11434**
   Ollama is running
4. **Airflow 3 의 헬스 주소는 /api/v2/monitor/health**
   2 시절의 /health 는 이 판에 없다

하나라도 안 되면

여기서 해결 후 진행

2단계

rag_pipeline_final — 다섯 태스크

collect	validate	build_index	prune	report
투입구 .txt → raw-docs/incoming/	20자 미만이면 raise ValueError	build_kb.main() + ingested_at	Lake 에 없는 조각을 지운다	lecture_kb 조각 수 출력

schedule="@daily" · catchup=False · default_args={"retries": 2, "retry_delay": timedelta(minutes=1)}

8-3 의 rag_pipeline 에서 바꾸는 것 셋 — 매일 돌고(@daily) · 1분 쉬고 두 번까지 재시도 · 조각마다 적재 시각

2단계 · 핵심 줄

적재 시각을 남기고, 바로 읽히게 하세요

적재 시각	now = datetime.now(timezone.utc).isoformat(timespec="seconds")
payload 만 덧붙임	qc.set_payload(COLLECTION, payload={"ingested_at": now}, points=FilterSelector(filter=Filter()))
왜 전체에	build_kb 가 방금 모든 문서를 다시 넣었다 — 벡터는 두고 값만
바로 읽히게	af dags reserialize — 기다리면 최대 5분
임포트 오류	af dags list-import-errors → No data found 면 통과
다음 실행	af dags next-execution rag_pipeline_final → 다음 자정(UTC)

웹 UI 의 rag_pipeline_final 상세에 @daily 와 태스크 다섯 — 아직 paused 라 자동으로 돌지는 않는다(검증 4단계에서 켠다)

3단계

챗봇을 최종본으로 — 8-4 app.py 에 둘을 더한다

1. **cat > app.py << 'EOF' ... EOF → streamlit run app.py**
   앱이 떠 있으면 Ctrl+C 로 끈 뒤 — ~/data-eng/week8 의 가상환경에서

2. **사이드바 — info = qdrant.get_collection(COLLECTION)**
   st.metric("조각 수", info.points_count) · 차원 info.config.params.vectors.size

3. **근거마다 — h.payload.get('ingested_at', '알 수 없음')**
   st.caption 에 적재 시각 — DAG 가 남긴 payload 키를 그 이름으로 읽는다

4. **접속 실패는 st.error("Qdrant 에 연결하지 못했습니다")**
   try/except — 역추적 대신 type(e).__name__ 한 줄

실측

조각 수 11

차원 768

3단계 · 결과

사이드바에 지식베이스가, 근거에는 아직 「알 수 없음」

지금 보이는 것

조각 수 11 · lecture_kb · 차원 768 · 모델 경로 ollama

- OpenAI 경로는 1536 · openai
- 검색할 근거 개수 슬라이더 1~10 (기본 3)
- 「대화 지우기」 → messages 를 비우고 st.rerun()

VS

아직 안 보이는 것

근거의 적재 시각은 「알 수 없음」

- 최종 DAG 를 아직 돌리지 않았다
- 검증 실습 5단계에서 시각이 찍힌다
- 지금은 정상 — 고칠 것이 아니다

4단계

이름 세 곳을 대조하세요 — lecture_kb

1 **grep -n "lecture_kb" dags/build_kb.py week8/app.py**
두 파일 모두 COLLECTION = os.getenv("COLLECTION", "lecture_kb")

2 **curl -s localhost:6333/collections | grep -o '"name":...'**
실제 Qdrant 목록에 "name":"lecture_kb"

3 **세 곳의 이름이 완전히 같아야 한다**
오타 한 글자로 「DAG 는 성공했는데 챗봇이 모른다」가 된다 — 이 대조를 습관으로

세 곳

build_kb.py

app.py · Qdrant

5단계

차원도 대조하세요 — 768 = 768

1. **curl -s .../collections/lecture_kb | grep -o '"size":[0-9]*'**
   "size":768 — 컬렉션이 받는 벡터 길이

2. **python -c "... len(get_emb().embed_query('차원'))"**
   질문 임베딩 차원 768 — 챗봇이 만드는 벡터 길이(OpenAI 경로는 둘 다 1536)

3. **다르면 챗봇과 DAG 가 다른 모델 경로를 쓰는 것**
   7-5 의 vector-schema.yaml 이 지키려던 바로 그 약속

기준

size 768

= 질문 768

확인 포인트

이 1.5시간이 끝날 때 확인할 것

1. health 넷이 healthy · collections 에 lecture_kb · Ollama is running
2. import 오류 No data found · next-execution 이 다음 자정(UTC)
3. 사이드바 조각 수 11 · 차원 768 · 근거의 적재 시각은 「알 수 없음」
4. build_kb.py · app.py · Qdrant 세 곳이 모두 lecture_kb
5. "size":768 과 질문 임베딩 차원 768 이 같다

최종 DAG 는 아직 paused — 다음 1H 에서 켜고, 새 문서 하나로 전 구간을 증명합니다

8-5 · 실습 1.5H 마무리

다음은 엔드투엔드 검증·시연 (1H)

대조군을 먼저 확인하고 lakehouse.txt 를 넣어 DAG 를 돌린 뒤, 챗봇이 그 내용으로 답하는지 봅니다

- ✓ 기준선 count 11 · 「레이크하우스가 뭐야?」 0.253
- ✓ 투입 → trigger → 12 · 0.517
- ✓ 두 번 돌려 12 그대로 · Qdrant 내려 up_for_retry
- ✓ submit/ 일곱 파일 · grep "sk-"

가져갈 것

paused 최종 DAG · 최종 app.py